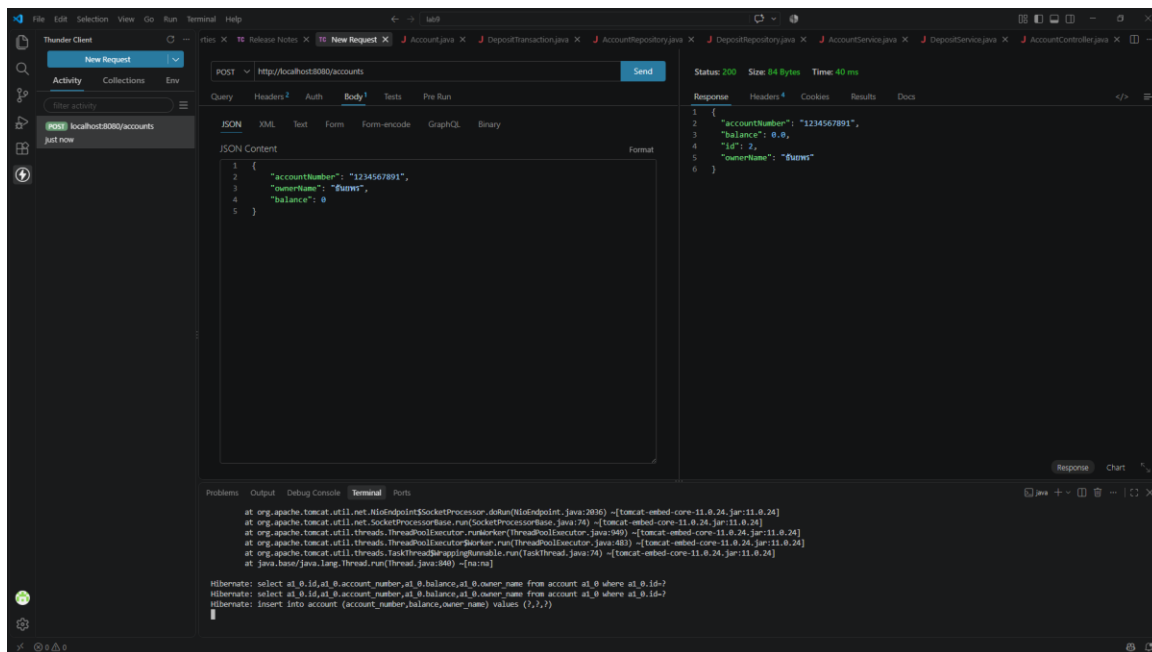
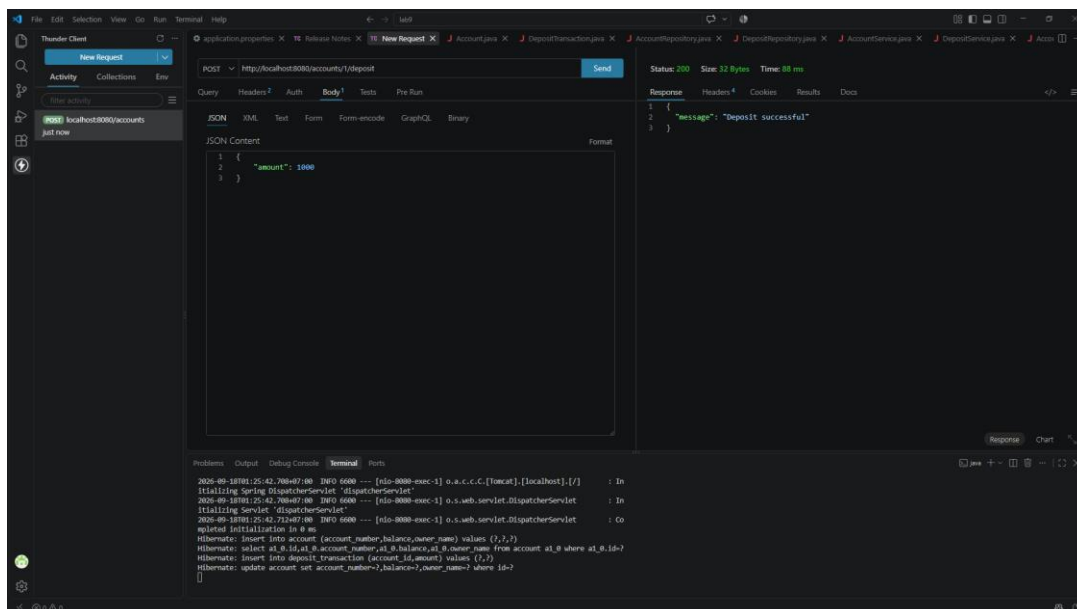


1. การสร้าง Account



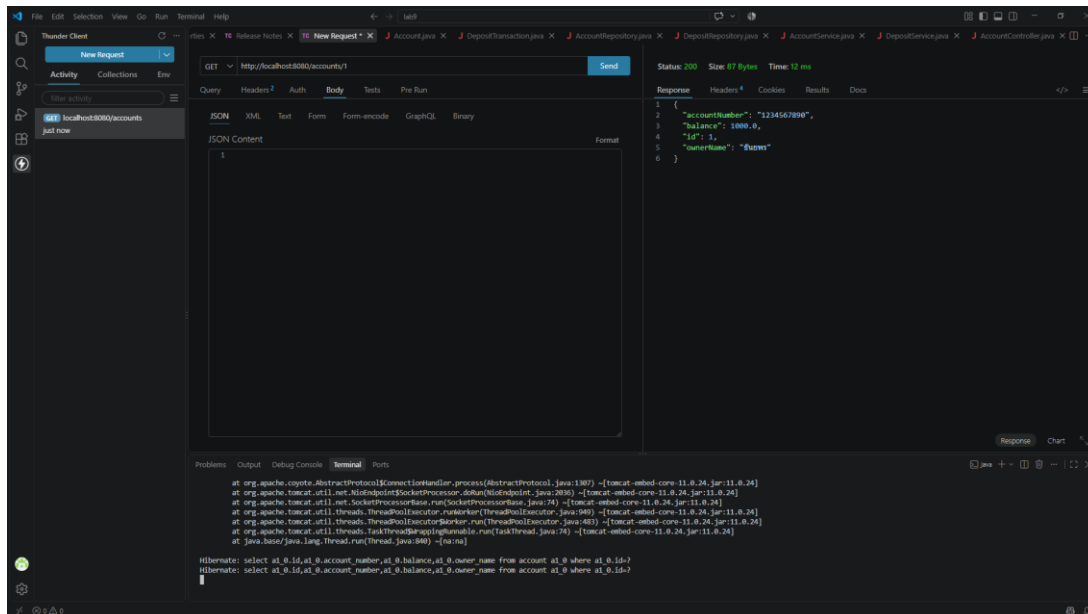
ยิง API แบบ POST ไปที่ `/accounts` เพื่อสร้างบัญชีใหม่ของ "ธันยพร" โดยกำหนดเงินเริ่มต้นไว้ที่ 0 บาท ระบบตอบกลับมาเป็น Status 200 OK พร้อมคืนข้อมูลบัญชี id: 1 ที่สร้างเสร็จแล้วกลับมาให้ แสดงว่าระบบทำงานถูกต้องและบันทึกข้อมูลลงตาราง `account` เรียบร้อย

2. การฝากเงินสำเร็จ



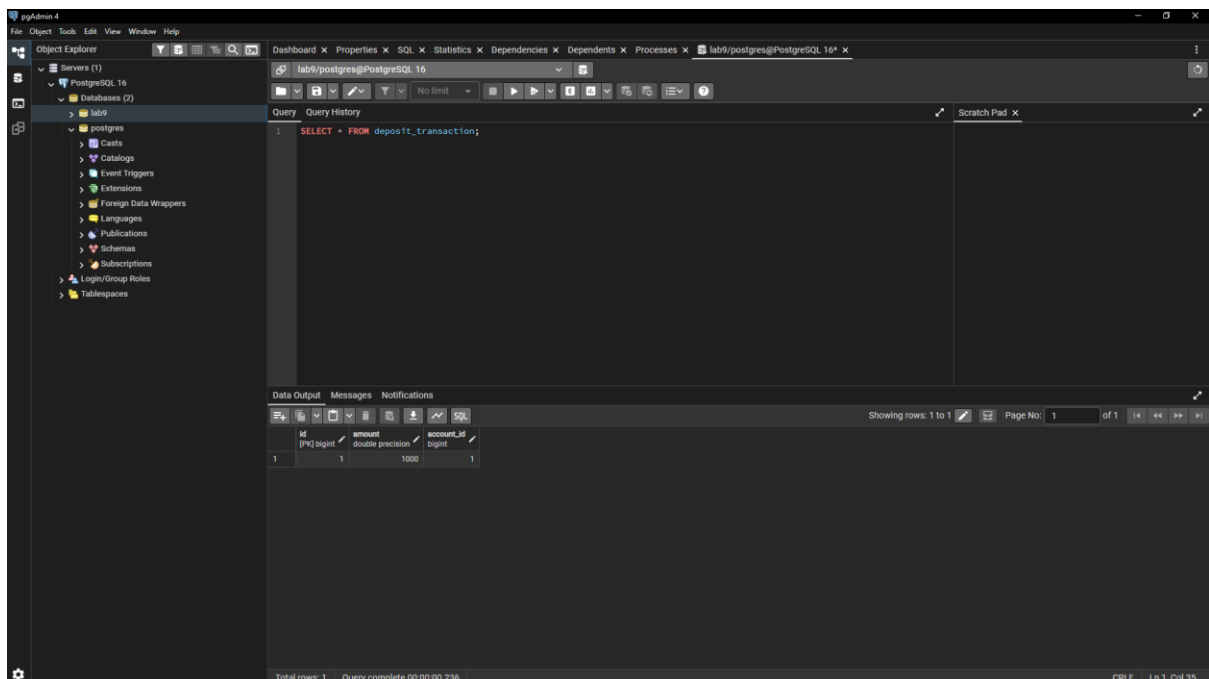
ยิง API แบบ POST ไปที่ `/accounts/1/deposit` เพื่อลองฝากเงินจำนวน 1,000 บาท ผลลัพธ์ฝั่งขวาแสดง Status 200 OK และมีข้อความตอบกลับมาว่า "Deposit successful" แสดงว่าระบบทำการปรับปรุงยอดเงินและบันทึกประวัติการฝากเงินสำเร็จ โดยไม่มี Error เกิดขึ้นระหว่างทำงาน

3. ตรวจสอบ Account



ยิง API แบบ GET ไปที่ `/accounts/1` เพื่อเช็คข้อมูลล่าสุดของบัญชีหลังฝากเงิน ผลลัพธ์แสดง Status 200 OK และยอดเงินคงเหลือ (balance) อัปเดตจาก 0.0 เพิ่มขึ้นเป็น 1000.0 บาท ยืนยันว่าระบบคำนวณและอัปเดตยอดเงินในบัญชีได้ถูกต้อง

4. ตรวจสอบ DepositTransaction



เปิดโปรแกรม pgAdmin 4 แล้วรันคำสั่ง SQL เพื่อตรวจสอบข้อมูลในตาราง `deposit_transaction` ผลปรากฏว่ามีประวัติการฝากเงินบันทึกอยู่ 1 รายการ ยอดเงิน 1000 บาท ผูกกับบัญชี id: 1 ยืนยันว่าระบบบันทึกประวัติลงฐานข้อมูลจริง

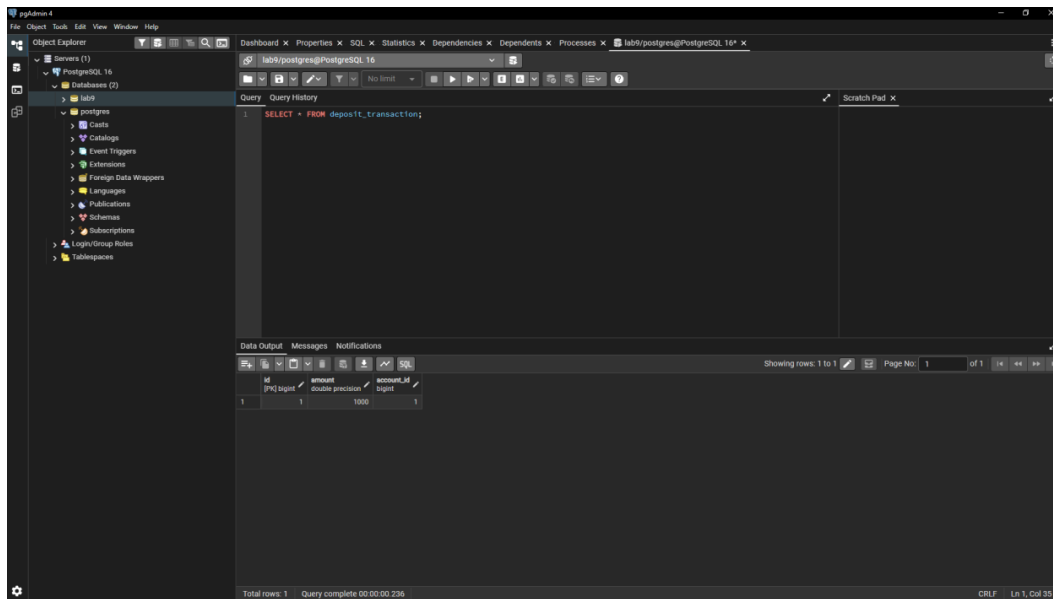
กันยพร เสนาโนฤทธิ 673380587-6 sec 3

5. ทดลอง Rollback

The screenshot shows the Thunder Client interface. A POST request is configured to `http://localhost:8080/accounts/1/deposit` with a JSON body: `{ "amount": 500 }`. The response status is 500 (Internal Server Error) with a size of 114 bytes and a time of 305 ms. The response body is: `{ "timestamp": "2026-09-17T18:34:14.673Z", "status": 500, "error": "Internal Server Error", "path": "/accounts/1/deposit" }`. The terminal shows the application running on Tomcat.

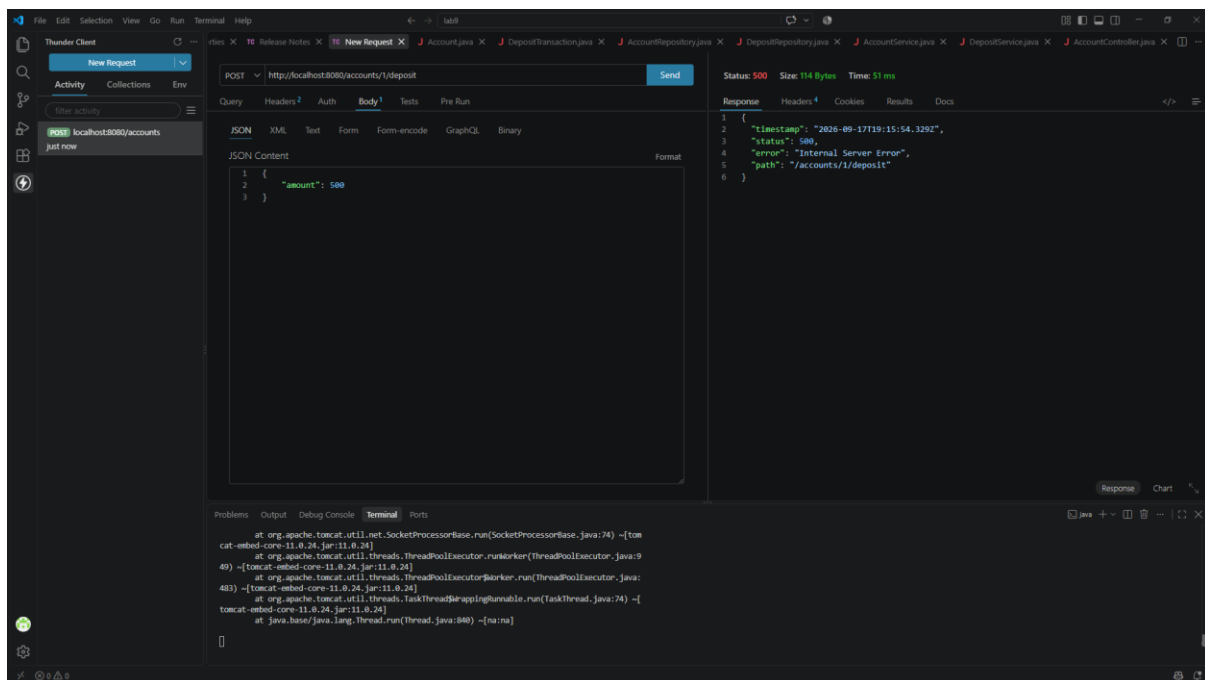
The screenshot shows the Thunder Client interface. A GET request is configured to `http://localhost:8080/accounts/1`. The response status is 200 (OK) with a size of 87 bytes and a time of 12 ms. The response body is: `{ "accountNumber": "1234567890", "balance": 1000.0, "id": 1, "ownerName": "Sutwas" }`. The terminal shows the application running on Tomcat and the Hibernate SQL query: `select a1.id,a1.account_number,a1_0.balance,a1_0.owner_name from account a1_0 where a1_0.id=?`.

ธันยพร เสนาโนฤทธิ 673380587-6 sec 3



ใส่โค้ดจำลอง Error ใน DepositService แล้วลองยิงฝากเงินเพิ่มอีก 500 บาท พบว่าระบบคืนค่า Status 500 Internal Server Error และเมื่อกลับไปเช็คยอดเงินในบัญชี ยอดเงินยังคงเป็น 1000.0 บาทเท่าเดิม เงิน 500 บาทไม่ได้ถูกบวกเพิ่ม แสดงว่า @Transactional สั่ง Rollback ยกเลิกการทำรายการทั้งหมดทันทีที่เกิด Error

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional



กันยพร เสนาโนฤทธิ 673380587-6 sec 3

The top screenshot shows an IDE with a REST client interface. A GET request to `http://localhost:8080/accounts/1` is shown. The response is a JSON object: `{ "accountNumber": "1234567890", "balance": 1500.0, "id": 1, "ownerName": "Bumms" }`. The bottom screenshot shows a PostgreSQL database view. The query `SELECT * FROM deposit_transaction;` is executed, showing two rows of data:

id	amount	account_id
1	1000	1
2	500	1

กรณีมี **@Transactional**: เมื่อเกิด Error ขึ้นกลางทาง ระบบจะทำการ Rollback ย้อนคืนข้อมูลทั้งหมดกลับไปสู่สภาพเดิม ยอดเงินในบัญชีจึงยังคงเป็น 1000.0 บาทเท่าเดิม ข้อมูลปลอดภัย ไม่ขัดแย้งกัน

กรณีไม่มี **@Transactional**: เมื่อเกิด Error ขึ้นกลางทาง คำสั่ง `save()` ที่ทำงานไปก่อนหน้าจะถูก Commit บันทึกลง PostgreSQL ทันที ยอดเงินในบัญชีจึงเพิ่มขึ้นเป็น 1500.0 บาท แม้รายการฝากเงินจะล้มเหลว ทำให้ข้อมูลในระบบเกิดความขัดแย้งกัน ยอดเงินเพิ่ม แต่ไม่มีประวัติบันทึก

7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

ข้อ 1: @Transactional ใน Spring Boot ทำหน้าที่อะไร?

ทำหน้าที่คอยควบคุมการทำงานกับฐานข้อมูลให้เป็นชุดเดียวกันครับ ถ้าทุกคำสั่งทำงานผ่านหมด ระบบจะ **Commit** บันทึกข้อมูลจริงลงฐานข้อมูล แต่ถ้าเกิด **Error** ขึ้นกลางทาง ระบบจะ **Rollback** ย้อนคืนค่าเดิมทั้งหมดทันที เพื่อป้องกันข้อมูลเสียหาย

ข้อ 2: ทำไมระบบฝาก-ถอนเงินจึงจำเป็นต้องใช้ @Transactional?

เพราะการฝาก-ถอนเงินมีหลายขั้นตอนต่อเนื่องกัน เช่น การอัปเดตยอดเงินในบัญชีกับการบันทึกประวัติ **Transaction** การใช้ **@Transactional** จะช่วยการันตีว่าต้องสำเร็จพร้อมกันทั้งคู่ หากขั้นตอนหลังเกิด **Error** เงินที่บวกเพิ่มไปแล้วก็จะถูกดึงกลับทันที เงินจะไม่เข้าฟรีโดยไม่มีความหมาย

ข้อ 3: Exception ประเภทใดบ้างที่ Spring จะทำการ Rollback ให้โดยอัตโนมัติ?

โดยค่าเริ่มต้น (Default) Spring จะ Rollback ให้อัตโนมัติเมื่อเกิด **Unchecked Exception** ครับ ได้แก่ พวก **RuntimeException** ทั้งหมด (เช่น **NullPointerException**, **IllegalArgumentException**) และกลุ่ม **Error** ในภาษา **Java**

ข้อ 4: หากเกิด RuntimeException ขึ้นหลังสั่ง save() บันทึกข้อมูลไปแล้ว ข้อมูลจะถูกบันทึกลง PostgreSQL หรือไม่ เพราะเหตุใด?

ไม่ถูกบันทึก เพราะคำสั่ง **save()** ของ **JPA** เป็นเพียงการเตรียมข้อมูลไว้ใน **Persistence Context** ยังไม่ได้ **Commit** จริง พอระบบเจอ **RuntimeException** ถัดมา Spring จะสั่ง **Rollback** ยกเลิกสิ่งที่เตรียมไว้ทั้งหมด ทำให้ไม่มีข้อมูลหลุดไปลง **PostgreSQL**

ข้อ 5: หากถอดคำสั่ง @Transactional ออกจาก DepositService ผลลัพธ์ขณะเกิด Error จะแตกต่างอย่างไร?

หากถอดออก คำสั่งที่ส่งรันไปก่อนหน้า เช่น บรทัดบวกยอดเงิน จะถูกบันทึกลงฐานข้อมูลจริงทันที และพอไปเจอ **Error** ในบรทัดถัดมา ยอดเงินจะเพิ่มขึ้นไปแล้วแต่ไม่มีประวัติการฝากบันทึกไว้ ส่งผลให้ข้อมูลในฐานข้อมูลขัดแย้งกัน

ข้อ 6: การระบุ @Transactional ไว้ที่ระดับ Class กับระดับ Method แตกต่างกันอย่างใด?

- ระดับ **Class**: คุณทุกเมธอดที่เป็น **public** ในคลาสนั้นให้ใช้เงื่อนไข **Transaction** เดียวกันทั้งหมด
- ระดับ **Method**: คุณเฉพาะเมธอดที่ระบุไว้เท่านั้น ทำให้เราปรับแต่งการทำงานเฉพาะเจาะจงได้ เช่น ตั้งค่า **readOnly = true** สำหรับเมธอดที่ใช้ดึงข้อมูลอย่างเดียวครับ